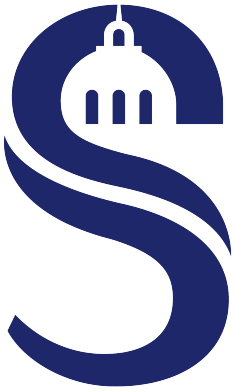
Thesis Title

Institution Name



SORBONNE UNIVERSITÉ

Author Name

Day Month Year

Abstract

Abstract goes here

Dedication

To mum and dad

Declaration

I declare that..

Acknowledgements

I want to thank...

Contents

Chapter 1

Introduction

1.1 Reactive Systems

When we think of computers, we usually picture desktops, laptops, or maybe smartphones. But our interaction with computer-based systems goes far beyond these familiar devices. From the automatic doors we pass through to the vehicles we drive, these are examples of systems that react automatically to real-time information, unnoticed. They are therefore naturally referred to as reactive systems.

Reactive systems are designed to respond immediately to changes in their environment. These systems are constantly monitoring and adjusting based on input—whether it is the presence of a human, the throttle pedal pressure, or the flow of data in an online service. The goal is simple to state: ensure a timely response to any changes or events. The criticity of this goal depends on the system, from very low for an online shopping website to very high for vehicles, requiring specific programming methods.

Programming a reactive system is complex and error-prone. Developers have to design code capable of handling all situations in a limited time and with limited resources (a computer's memory is physically finite). Dedicated tools are designed to facilitate this process and make it more reliable. But while reactivity is a common goal, other priorities may differ: a website expects fast response times on average rather than reliability, while a vehicle guarantees a an execution time bellow a worst-case value.

This thesis focuses on the programming of critical reactive systems, and more specifically on vehicles. The work was conducted at Ampere, the software branch of the Renault Group, which is facing issues with its current programming method.

1.2 Challenges at Ampere

1.3 Paradigm Shift

1.4 Domain Specific Language

1.5 TODO ou Mini plan

- reactive systems
- objectifs diminussion des calculs et des messages sur le bus
- changement de paradigme : de "périodique" à "onchange"
- DSL Rust

Chapter 2

State of the art

- FRP (+ ratus)
- synchrone
- syntaxe + semantics operational + denotational semantics

Chapter 3

GRust

- syntaxe
- type system
- examples
- benchmark (diminussion des comms sur le bus)

3.1 GRust syntax

The GRust modeling language includes a component-modeling kernel GRsync, a specification language GReusot, an interface GRfrp along with a library GReact and syntactic sugars that simplify system modeling. GReusot is a wrapper for Pearlite, the specification language of Creusot [6].

GRust example: Urban braking in the event of pedestrians

```
import signal float   car.speed_km_h;
import event  float   car.detect.left.pedestrian as l;
import event  float   car.detect.right.pedestrian as r;
export signal Braking car.urban.braking.brakes;
service urban_braking {
   event float pedestrian = merge(l, r);
   event unit timeout_pedest = timeout(pedestrian, 500);
   brakes = braking(pedestrian, timeout_pedest, speed_km_h);
}
function brakes(distance: float, speed: float) {
   let D: float = compute_soft_braking_distance(speed);
   return if D < distance then SoftBrake else UrgentBrake;
}
component braking(pedest: float?, timer: unit?, speed: float) -> (state: Braking)
    requires { 0. <= speed && speed < 50. }  // urban limit
    ensures  { pedest? => state != NoBrake } // safety
{
    state = when {
        init    => NoBrake,
        pedest? => brakes(pedest, speed),
        timer?  => NoBrake,
    };
}
```

The example above implements an urban braking system that triggers soft or urgent braking when a pedestrian is detected. The function `brakes` and the component `braking` model functional requirements as a finite-state machine in GRsync. To ensure correct behavior, `braking` includes GReusot specifications: if `speed` respects the speed limit then pedestrian detection triggers braking. The GRfrp interface `urban_braking` links the component instantiation in the service to real system data flows. It uses the GReact library to combine left and right pedestrian detection (with `merge`) and to create a wake-up delay to the event (with `timeout`).

3.1.1 GRsync language

Lustre nodes bridge the gap between functional requirements and their implementation. As others (SCADE [5], Zélus [2], Heptagon [3], etc), we chose this language as the core of GRsync.

Syntax. 1: GRsync without GReusot

Identifiers	f, x	$\in$	*Ident*
Constants	c	$\in$	*Const*
n-ary operators	op	$\in$	*Oper*
Kinds	k	$::=$	$C \mid F$
Declarations	$\mathcal{D}$	$::=$	$\texttt{let}\ k\ f(x_i)^i = e^k \mid \mathcal{D}\ ;\ \mathcal{D}$
Functional expressions	e^F	$::=$	$c \mid x \mid op(e_i^F)^i \mid (e_i^F)^i \mid f(e_i^F)^i \mid \texttt{let}\ \mathcal{E}^F\ \texttt{in}\ e^F$
Component expressions	e^C	$::=$	$e^F \mid \texttt{let}\ \mathcal{E}^C\ \texttt{in}\ e^C$
Memory expressions	e^M	$::=$	$e^F \mid \texttt{last}\ x$
Patterns	pat	$::=$	$x \mid (pat_i)^i$
Event patterns	$epat$	$::=$	$\texttt{let}\ pat = x? \mid e^F \mid (epat_i)^i$
Functional equations	$\mathcal{E}^F$	$::=$	$pat = e^F \mid \mathcal{E}_1^F\ \texttt{and}\ \mathcal{E}_2^F \mid \texttt{match}\ e^F\ \{\ pat_i\ \texttt{if}\ e_i^F\ \texttt{=>}\ \mathcal{E}_i^F\ \}$
Component equations	$\mathcal{E}^C$	$::=$	$\mathcal{E}^F \mid pat = e^C \mid \mathcal{E}_1^C\ \texttt{and}\ \mathcal{E}_2^C \mid \texttt{init}\ \mathcal{E}^I\ \texttt{in}\ \mathcal{E}^C$
		$\mid$	$\texttt{when}\ \{\ \texttt{init}\ \texttt{=>}\ \mathcal{E}^I,\ epat_i\ \texttt{if}\ e_i^M\ \texttt{=>}\ \mathcal{E}_i^R\ \}$
Reactive equations	$\mathcal{E}^R$	$::=$	$\mathcal{E}^F \mid pat = e^M \mid pat = \texttt{emit}\ e^M \mid \mathcal{E}_1^R\ \texttt{and}\ \mathcal{E}_2^R$

The syntax of GRsync is presented in Syntax 1, for sake of clarity we first omit the GReusot specification.

Declaration

Software components are modeled as stream functions ($\texttt{let}\ k\ f(x_i)^i = e^k$) called nodes, where k is the node's kind, $(x_i)^i$ is the tuple of input streams, and e is the output stream expression. The kinds of nodes can be C for component, nodes with finite buffers, or F for functions, memory-free nodes. [1].

Expression

Expressions are decomposed in three kinds: functional expressions that are memory-free expressions, component expressions that can contain local declarations of component equations, and memory expressions. Functional expressions are composed of constants (c), identifiers (x), n-ary mathematical operations ($op(e_i^F)^i$), tuples ($(e_i^F)^i$), memory-free node application ($f(e_i)^i$), and local declarations of functional equations ($\texttt{let}\ \mathcal{E}^F\ \texttt{in}\ e^F$). Component expressions add to functional expressions the local declaration of component equations ($\texttt{let}\ \mathcal{E}^C\ \texttt{in}\ e^C$). Memory expressions add to functional expressions a buffered stream ($\texttt{last}\ x$).

Equation

Similarly to expressions, equations are divided in three categories: functional equations that are memory free, component equations that include memory initialization and event handling `when` equations, and reactive equations that are triggered by events. A functional equation can be a definition of streams ($pat = e^F$), simultaneous equations ($\mathcal{E}_1^F\ \texttt{and}\ \mathcal{E}_2^F$), or a pattern matching set of equations ($\texttt{match}\ e^F\ \{\ pat_i\ \texttt{if}\ e_i^F\ \texttt{=>}\ \mathcal{E}_i^F\ \}$). A component equation can

[1] In Example 1, C and F are replaced by `component` and `function` respectively.

be a functional equation, a definition of streams using a component equation ($pat = e^C$), simultaneous component equations ($\mathcal{E}_1^C$ `and` $\mathcal{E}_2^C$), initializations of buffers (`init` $\mathcal{E}^I$ `in` $\mathcal{E}^C$), or an event handler (`when` { `init =>` $\mathcal{E}^I$, $epat_i$ `if` e_i^M `=>` $\mathcal{E}_i^R$ }). A reactive equation can be a functional equation, a definition of streams using a memory equation ($pat = e^C$), an event emission ($pat =$ `emit` e^M), or simultaneous reactive equations ($\mathcal{E}_1^R$ `and` $\mathcal{E}_2^R$).

Event detection pattern

An event detection pattern can either be a test for the presence of an event (`let` $pat = x?$) where the value of x is stored in the patter pat if present, a rising edge detection from a boolean functional expression (e^F), or multiple events ($(epat_i)^i$).

Comparision

GRSYNC syntax does not contain clocks, which require a fine understanding of their semantics to be used properly. Clocks are used implicitly in the semantics of `match` and `when` expressions. We believe it is reasonable to expect (`match` e^F { pat_i `=>` $\mathcal{E}^C$ }) to only trigger ($\mathcal{E}^C$) when (e^F) matches (pat_i) ; (`when`$_e$ { pat_i^R `=>` e_i^C }) only evaluate to (e_i^C) when (pat_i^R) appears ; (`when`$_s$ { pat_i^R `=>` e_i^C }) only update its value to (e_i^C) when (pat_i^R) appears.

3.1.2 GRfrp language

GRFRP is an interface language used to model the interaction between real system data flows and GRSYNC state machines.

Syntax. 2: GRFRP

Identifiers	s, f, x	$\in$	*Ident*
Constants	Δ	$\in$	*Const*
Time	T	$\in$	*Time*
Kinds	k	$::=$	`signal` | `event`
Declarations	$\mathcal{D}$	$::=$	`service` s $\{(\mathcal{S}_i)^i\}$ | $\mathcal{D}$ $\mathcal{D}$
Patterns	pat	$::=$	k x | $(pat_i)^i$
Flow statements	$\mathcal{S}$	$::=$	`let` $pat = \sigma$; | `import` k x ; | `export` k x ;
Flows	σ	$::=$	x | $f(\sigma_i)^i$ | `time` | `sample` σ T | `scan` σ T | `timeout` σ T
		|	`throttle` σ Δ | `onchange` σ | `persist` σ | `merge` σ σ [5]

GRFRP syntax is sketched in Syntax 2.

Service declaration

Services are defined as a set of flow statements (`service` s $\{(\mathcal{S}_i)^i\}$).

[5] `merge` refers to the FRP operator, different from the LUSTRE operator.

Flow statement

In a service, system flows can be imported (`import` $k\ x$;) or exported (`export` $k\ x$;) and local flows can be defined (`let` $pat = \sigma$;), where pat is a pattern of flows.

A pattern of flows can be a single flow ($k\ x$), where k is the kind of flow, or a tuple of patterns ($(pat_i)^i$). Two kinds of flow are considered: `signal` representing continuous values varying over time ; `event` representing punctual values occurring at any time.

Flow expression

Flows are defined by flow expressions: another flow call (x), GRSYNC node applications ($f(\overline{\sigma})$), or GREACT operations.

GREACT operations create time constrained flows. It provides the current time of execution (`time`), makes it possible to look for the occurence of an event periodically (`sample` $\sigma\ T$) or for the change of a signal periodically (`scan` $\sigma\ T$), to create an event timer (`timeout` $\sigma\ T$) that indicates the absence of event σ for a period T. It can also filter a signal σ to pass values that have at least a difference Δ from the previous one (`throttle` $\sigma\ \Delta$), create an event of changes from a signal (`onchange` σ), create a signal from the continuation of an event (`persist` σ), and combine two events in one (`merge` $\sigma\ \sigma$).

Comparision

Usual synchronous languages implementations interface state machines in a periodic pull manner. Defining a suitable base clock is complex: it must be fast enough to neglect processing delays [7] and capture all major events from sensors, but slow enough for the synchronous hypothesis to hold. Push-based systems update outputs as soon as possible, the rate of inputs defines the base clock.

Chapter 4

Semantics

The GRust programming language has two main constructions, referred to as components and services, with two different purpose and semantics.

A component represents a control unit that computes output flows from received input flows. Those flows, referred to as streams, are synchronous: all streams in one component have a value at the same instants.Components are similar to nodes in Lustre [8], without explicit clocks from the user. Its implementation in GRust defines the values of the output streams at one instant according to the previous and current values of the input streams.

A service is a specific feature of the vehicle system that also computes output flows from received input flows. Those flows, referred to as signals and events, are asynchronous: the service do not receive values from all signals and events at the same time. A signal always holds a value that can change over time, whereas an event is a punctual value that may arrive at any time. The implementation of a service in GRust defines the output signals and events according to the input signals and events. To do so, the service may call some components or the operators of GRfrp.

4.1 Operational Semantics

We will first present the operational semantics of components and then the semantics of services.

4.1.1 State-Machine Semantics

A node computes the current output values based on both the current and past input values. It operates like a state machine, where its state represents all the values needed for future computations: this includes the values of streams referenced with `last` s and the states of other nodes it invokes. The state has a finite size, determined by the number of streams using `last` s_i and the number of called nodes.

Representing the operational semantics of a node as a finite-state machine is a common practice in Lustre-like languages [4]. We denote the transition function of a node f, defined in a program G, from a state s receiving input values $(v_i)^i$ to a new state s' producing output values $(v_o)^o$ as $G \vdash s, (v_i)^i \xrightarrow[f]{step} s', (v_o)^o$.

Stream expressions and equations used in the definition of a node can also be viewed as a state machine, but with a context ρ replacing the input values and involving different types

of outputs. The transition of an expression e in context ρ, starting from state s, produces a next state s' and a value v, represented as $G, \rho \vdash s \xrightarrow[e]{step} s', v$. Similarly, the transition of an equation $\mathcal{E}$ from state s results in a next state s' and an updated context ρ', and is written as $G, \rho \vdash s \xrightarrow[\mathcal{E}]{step} s', \rho'$.

Node

The rule (Comp) defines the node transition, which performs the transition of the output stream expression with a context ρ initialized with the input values $(v_i)^i$ given to the node.

$$\boxed{G \vdash s, (v_i)^i \xrightarrow[f]{step} s', (v_o)^o}$$

$$\frac{G[f] = \texttt{let}\ k\ f(x_i)^i = e^k \qquad G, [v_i/x_i]^i \vdash s \xrightarrow[e^k]{step} s', (v_o)^o}{G \vdash s, (v_i)^i \xrightarrow[f]{step} s', (v_o)^o}\ \text{(Comp)}$$

Expression

The rules for constant (Const) and identifier (Ident) expressions use an empty state $()$. They produce respectively the constant value and the value stored in the context ρ.

For the memory expression `last` x, the state is also empty. The memory for the previous value of x is managed by the initialization equation `init` $x = c$ `in` $\mathcal{E}$, which stores the last value of x in the context ρ. This approach ensures there is only one storage for the last value of x. Without this, each `last` expression would require its own separate state. The rule for the `last` expression (Last) retrieves the memory of the previous value of x from the context ρ.

For operation expressions (Oper) and tuple expressions (Tuple), the rules propagate state transitions while combining the outputs appropriately.

When transitioning an expression that uses a local equation (Let), the process requires states for both the equation and the expression. The transition first applies to the equation, producing an updated context. This updated context is then used to perform the transition for the expression.

For node applications (App),the process begins by transitioning the input expressions. The resulting inputs are then used in the transition function of the node.

$$\boxed{G, \rho \vdash s \xrightarrow[e]{step} s', v}$$

$$\frac{}{G, \rho \vdash () \xrightarrow[c]{step} (), c}\ \text{(Const)} \qquad \frac{}{G, \rho \vdash () \xrightarrow[x]{step} (), \rho[x]}\ \text{(Ident)}$$

$$\frac{}{G, \rho \vdash () \xrightarrow[\texttt{last}\ x]{step} (), \rho[\texttt{last}\ x]}\ \text{(Last)} \qquad \frac{\forall i \quad G, \rho \vdash s_i \xrightarrow[e_i]{step} s'_i, v_i}{G, \rho \vdash (s_i)^i \xrightarrow[op(e_i)^i]{step} (s'_i)^i, op(v_i)^i}\ \text{(Oper)}$$

$\boxed{G, \rho \vdash s \xrightarrow[e]{step} s', v}$ *continued*

$$\frac{\forall i \quad G, \rho \vdash s_i \xrightarrow[e_i]{step} s'_i, v_i}{G, \rho \vdash (s_i)^i \xrightarrow[(e_i)^i]{step} (s'_i)^i, (v_i)^i} \quad (\textsc{Tuple}) \qquad \frac{\begin{array}{c}\forall i \quad G, \rho \vdash s_i \xrightarrow[e_i]{step} s'_i, v_i \\ G \vdash s_f, (v_i)^i \xrightarrow[f]{step} s'_f, (v_o)^o\end{array}}{G, \rho \vdash (s_f, (s_i)^i) \xrightarrow[f(e_i)^i]{step} (s'_f, (s'_i)^i), (v_o)^o} \quad (\textsc{App})$$

$$\frac{\begin{array}{c}G, \rho \vdash s \xrightarrow[\mathcal{E}]{step} s', \rho' \\ G, \rho' \vdash s_e \xrightarrow[e]{step} s'_e, v\end{array}}{G, \rho \vdash (s, s_e) \xrightarrow[\texttt{let } \mathcal{E} \texttt{ in } e]{step} (s', s'_e), v} \quad (\textsc{Let})$$

Equation

The rule for the declaration of a pattern of streams (Pat) performs the transition of the expression and updates the values of defined identifiers in the context. The function `varsPat` helps to retrieve the identifiers from the pattern in the proper order.

$\boxed{\texttt{varsPat } pat \ : \ \mathcal{P}(Ident)}$

$$\begin{array}{c}\texttt{varsPat } x = x \\ \texttt{varsPat } (pat_i)^i = (\texttt{varsPat } pat_i)^i\end{array}$$

The initialization of $\texttt{last}x_i$ memories handles their updates within the context ρ at each step, as described in rule (Init). It performs the step transition of the equation where the initializations are needed, takes the resulting updated context ρ', and stores in its state the current values of x_i. This ensures that the values of x_i are correctly maintained and accessible for the next transition.

$\boxed{G, \rho \vdash s \xrightarrow[\mathcal{E}]{step} s', \rho'}$

$$\frac{\texttt{varsPat } pat = (x_i)^i \qquad G, \rho \vdash s \xrightarrow[e]{step} s', (v_i)^i}{G, \rho \vdash s \xrightarrow[pat=e]{step} s', \rho[v_i/x_i]^i} \quad (\textsc{Pat})$$

$$\frac{G, \rho[m_i/\texttt{last } x_i]^i \vdash s \xrightarrow[\mathcal{E}]{step} s', \rho' \qquad \forall i \quad m'_i = \rho'[x_i]}{G, \rho \vdash ((m_i)^i, s) \xrightarrow[\texttt{init } x_1=c_1 \texttt{ and } \ldots \texttt{ and } x_n=c_n \texttt{ in } \mathcal{E}]{step} ((m'_i)^i, s'), \rho' \backslash [x_i]^i} \quad (\textsc{Init})$$

$$\frac{G, \rho \vdash (s_1, ..., s_n) \xrightarrow[\mathcal{E}_1 \texttt{ and } \ldots \texttt{ and } \mathcal{E}_n]{fix\ n} \rho' \qquad \forall i \quad G, \rho' \vdash s_i \xrightarrow[\mathcal{E}_i]{step} s'_i, \rho'}{G, \rho \vdash (s_1, ..., s_n) \xrightarrow[\mathcal{E}_1 \texttt{ and } \ldots \texttt{ and } \mathcal{E}_n]{step} (s'_1, ..., s'_n), \rho'} \quad (\textsc{And})$$

A node may define output and local streams using synchronous equations $\mathcal{E}_1$ `and` ... `and` $\mathcal{E}_n$, with all $\mathcal{E}_i$ satisfied at the same time to ensure all streams are updated with their next values simultaneously. Before the transition of the synchronous equations, all the values they define are undetermined. The step function iteratively computes these undetermined values using the known ones until all values are resolved. The undetermined values are stored in the context as $\perp$ and any operation with this bottom value returns a bottom value. The semantics of synchronous equations (And) relies on the $fixk$ fixpoint transition, (Fix-0) and (Fix-k), which performs these iterative computations up to a maximum of k recursive calls.

$$\boxed{G, \rho \vdash (s_1, ..., s_n) \xrightarrow[\mathcal{E}_1 \texttt{ and ... and } \mathcal{E}_n]{fix\ n} \rho'}$$

$$\frac{}{G, \rho \vdash (s_1, ..., s_n) \xrightarrow[\mathcal{E}_1 \texttt{ and ... and } \mathcal{E}_n]{fix\ 0} \rho} \quad \text{(Fix-0)}$$

$$\frac{G, \rho \vdash (s_1, ..., s_n) \xrightarrow[\mathcal{E}_1 \texttt{ and ... and } \mathcal{E}_n]{fix\ k-1} \rho' \qquad k > 0 \qquad \forall i \quad G, \rho' \vdash s_i \xrightarrow[\mathcal{E}_i]{step} s_i', \rho_i}{G, \rho \vdash (s_1, ..., s_n) \xrightarrow[\mathcal{E}_1 \texttt{ and ... and } \mathcal{E}_n]{fix\ k} \cup_i \rho_i} \quad \text{(Fix-k)}$$

The semantics of `match` equations is described by three rules, corresponding to the three possible outcomes when matching a branch: the pattern does not match, the pattern matches but the guard evaluates to false, and the pattern matches with the guard evaluating to true. To determine whether a value matches a given pattern, the function `instanceOf` is used. This function returns the set of all values that can match the specified pattern.

$$\boxed{\texttt{instanceOf}\ pat\ :\ \mathcal{P}(\mathit{Value})}$$

$$\texttt{instanceOf}\ x = \texttt{typeOf}\ x$$
$$\texttt{instanceOf}\ (pat_i)^i = \Pi_i\ \texttt{instanceOf}\ pat_i$$

The rule (NotMatch) describes the case where the first pattern pat_0 does not match the values $(v_j)^j$ of the matching expression e, meaning $(v_j)^j$ is not included in `instanceOf`pat_0, and the other patterns pat_i have to be tested.

In the rule (GuardMatch), the first pattern pat_0 matches the values $(v_j)^j$ of the matching expression e, meaning $(v_j)^j$ is included in `instanceOf`pat_0, but the guard e_0 evaluates to false, and the other patterns pat_i have to be tested.

Finally, the rule (Match) shows that when the first pattern pat_0 matches the values $(v_j)^j$ of the matching expression e and the guard e_0 evaluates to true, it evaluates the equations $\mathcal{E}_0$ of the matching branch, and the other branches are not evaluated. In both rules (GuardMatch) and (Match), the pattern matching the values of e creates new identifiers that have to be added to the transition context of the guard and the branch equations.

$\boxed{G, \rho \vdash s \xrightarrow[\mathcal{E}]{step} s', \rho'}$ *match equation*

$$\frac{\begin{array}{c} G, \rho \vdash s \xrightarrow[e]{step} s', (v_j)^j \qquad (v_j)^j \notin \texttt{instanceOf}\ pat_0 \\ G, \rho \vdash (s, (s_i)^i) \xrightarrow[\texttt{match}\ e\ \{\ (pat_i\ \texttt{if}\ e_i\ \texttt{=>}\ \mathcal{E}_i)^i\ \}]{step} (s', (s'_i)^i), \rho' \end{array}}{G, \rho \vdash (s, s_0, (s_i)^i) \xrightarrow[\texttt{match}\ e\ \{\ pat_0\ \texttt{if}\ e_0\ \texttt{=>}\ \mathcal{E}_0\ ,\ (pat_i\ \texttt{if}\ e_i\ \texttt{=>}\ \mathcal{E}_i)^i\ \}]{step} (s', s_0, (s'_i)^i), \rho'} \quad (\textsc{NotMatch})$$

$$\frac{\begin{array}{c} G, \rho \vdash s \xrightarrow[e]{step} s', (v_j)^j \qquad (v_j)^j \in \texttt{instanceOf}\ pat_0 \qquad \texttt{varsPat}\ pat_0 = (x_j)^j \\ G, \rho[v_j/x_j]^j \vdash () \xrightarrow[e_0]{step} (), False \qquad G, \rho \vdash (s, (s_i)^i) \xrightarrow[\texttt{match}\ e\ \{\ (pat_i\ \texttt{if}\ e_i\ \texttt{=>}\ \mathcal{E}_i)^i\ \}]{step} (s', (s'_i)^i), \rho' \end{array}}{G, \rho \vdash (s, s_0, (s_i)^i) \xrightarrow[\texttt{match}\ e\ \{\ pat_0\ \texttt{if}\ e_0\ \texttt{=>}\ \mathcal{E}_0\ ,\ (pat_i\ \texttt{if}\ e_i\ \texttt{=>}\ \mathcal{E}_i)^i\ \}]{step} (s', s_0, (s'_i)^i), \rho'} \quad (\textsc{GuardMatch})$$

$$\frac{\begin{array}{c} G, \rho \vdash s \xrightarrow[e]{step} s', (v_j)^j \qquad (v_j)^j \in \texttt{instanceOf}\ pat_0 \qquad \texttt{varsPat}\ pat_0 = (x_j)^j \\ G, \rho[v_j/x_j]^j \vdash () \xrightarrow[e_0]{step} (), True \qquad G, \rho[v_j/x_j]^j \vdash s_0 \xrightarrow[\mathcal{E}_0]{step} s'_0, \rho' \end{array}}{G, \rho \vdash (s, s_0, (s_i)^i) \xrightarrow[\texttt{match}\ e\ \{\ pat_0\ \texttt{if}\ e_0\ \texttt{=>}\ \mathcal{E}_0\ ,\ (pat_i\ \texttt{if}\ e_i\ \texttt{=>}\ \mathcal{E}_i)^i\ \}]{step} (s', s'_0, (s_i)^i), \rho'} \quad (\textsc{Match})$$

The semantics of `when` equations differs from those of `match` equations. Event patterns involve stateful transitions, denoted $G, \rho \vdash s \xrightarrow[epat]{step} s', b, \rho'$, which must be performed at every transition of `when` equations. These transitions return three results: the updated state s' of the event pattern, a boolean b indicating whether the event is present, and an updated context ρ' that includes the values of identifiers corresponding to the present events. There are two semantics rules for `when` equations, one applies when an event pattern is triggered, and the other applies when all events are absent.

When an event patter $epat_k$ is the first to be present and its guard evaluates to true, the rule (When) applies. In this case, the equation $\mathcal{E}_k$ performs a transition. All signals x_j, initialized by $x_j = c_j$, not defined by $\mathcal{E}_k$ are added to the new context with their previous value. Similarly, for events y_l not defined by $\mathcal{E}_k$, a $None$ value is added to the updated context. The new values of signals are then stored in memory m'_j.

When all events are absent, the rule (NotWhen) applies. The previous value of all signals x_j are added to the new context, and a $None$ value for all events y_l.

$\boxed{G, \rho \vdash s \xrightarrow[\mathcal{E}]{step} s', \rho'}$ *when equation*

$$\frac{\begin{array}{c}\forall i \quad G, \rho \vdash s_i^p \xrightarrow[epat_i]{step} s_i^{p\prime}, b_i, \rho_i \qquad b_k = True \qquad G, \rho_k \vdash () \xrightarrow[e_k]{step} (), True \\ \forall i < k \quad b_i = False \vee (b_i = True \wedge G, \rho_i \vdash () \xrightarrow[e_i]{step} (), False) \\ G, \rho_k[m_j/\texttt{last}\ x_j]^j \vdash s_k \xrightarrow[\mathcal{E}_k]{step} s'_k, \rho'_k \qquad \forall i \neq k \quad s'_i = s_i \\ \{x_{j'}\}^{j'} = \{x_j | x_j \notin \texttt{varsEq}\ \mathcal{E}_k\} \qquad \{y_{l'}\}^{l'} = \{y_l | y_l \notin \texttt{varsEq}\ \mathcal{E}_k\} \qquad \forall j \quad m'_j = \rho'[x_j] \\ \cup_i \texttt{varsEq}\ \mathcal{E}_i = \{x_j\}^j \cup \{y_l\}^l \qquad \rho' = \rho'_k[None/y_{l'}]^{l'}[m_{j'}/x_{j'}]^{j'}\end{array}}{G, \rho \vdash ((m_j)^j, (s_i^p)^i, (s_i)^i) \xrightarrow[\texttt{when \{ init => } (x_j = c_j)^j, (epat_i\ \texttt{if}\ e_i\ \texttt{=>}\ \mathcal{E}_i)^i\ \texttt{\}}]{step} ((m'_j)^j, (s_i^{p\prime})^i, (s'_i)^i), \rho'} \quad (\textsc{When})$$

$$\frac{\begin{array}{c}\forall i \quad G, \rho \vdash s_i^p \xrightarrow[epat_i]{step} s_i^{p\prime}, b_i, \rho_i \\ \forall i \quad b_i = False \vee (b_i = True \wedge G, \rho_i \vdash () \xrightarrow[e_i]{step} (), False) \\ \cup_i \texttt{varsEq}\ \mathcal{E}_i = \{x_j\}^j \cup \{y_l\}^l \qquad \rho' = \rho[None/y_l]^l[m_j/x_j]^j\end{array}}{G, \rho \vdash ((m_j)^j, (s_i^p)^i, (s_i)^i) \xrightarrow[\texttt{when \{ init => } (x_j = c_j)^j, (epat_i\ \texttt{if}\ e_i\ \texttt{=>}\ \mathcal{E}_i)^i\ \texttt{\}}]{step} ((m_j)^j, (s_i^{p\prime})^i, (s_i)^i), \rho'} \quad (\textsc{NotWhen})$$

The last kind of equation is the event emission. Its semantics, as defined by rule (Emit), involve wrapping the value of e in a *Some* option. Event emissions are used within branches of `when` equations, meaning they transition only when the associated event is present. At all other times, their value is set to *None* in the context.

$\boxed{G, \rho \vdash s \xrightarrow[\mathcal{E}]{step} s', \rho'}$ *emit equation*

$$\frac{\texttt{varsPat}\ pat = (x_i)^i \qquad G, \rho \vdash s \xrightarrow[e]{step} s', (v_i)^i}{G, \rho \vdash s \xrightarrow[pat=\texttt{emit}\ e]{step} s', \rho[Some(v_i)/x_i]^i} \quad (\textsc{Emit})$$

Event detection pattern

As explained when describing the semantics of `when` equations, an event pattern performs transitions, denoted $G, \rho \vdash s \xrightarrow[epat]{step} s', b, \rho'$. These transitions return three results: the updated state s' of the event pattern, a boolean b indicating whether the event is present, and an updated context ρ' that includes the values of identifiers corresponding to the present events.

A rising edge on an expression e stores the previous boolean value of e in its state. It returns true when it detects a transition from false to true. This behavior is defined by the semantics described in rule (RisingEdge).

A tuple event pattern, as described in (Tuple), detects whether all events in the tuple are present.

The presence of an event `let` $pat = x?$ is determined by rule (Event), which checks if the identifier x is associated with a *Some* value in the context ρ. If so, the context is updated with the unwrapped values of the event, and the boolean b is set to true.

Conversely, if the identifier x is associated with a *None* value, rule (NoEvent) applies. In this case, the context remains unchanged, and the boolean b is set to false.

$$\boxed{G, \rho \vdash s \xrightarrow[epat]{step} s', b, \rho'}$$

$$\frac{G, \rho \vdash s \xrightarrow[e]{step} s', b}{G, \rho \vdash (m, s) \xrightarrow[e]{step} (b, s'), !m \wedge b, \rho} \quad (\textsc{RisingEdge})$$

$$\frac{\forall i \quad G, \rho \vdash s_i \xrightarrow[epat_i]{step} s'_i, b_i, \rho_i}{G, \rho \vdash (s_i)^i \xrightarrow[(epat_i)^i]{step} (s'_i)^i, \wedge_i b_i, \cup_i \rho_i} \quad (\textsc{Tuple}) \qquad \frac{None = \rho[x]}{G, \rho \vdash () \xrightarrow[\texttt{let } pat=x?]{step} (), False, \rho} \quad (\textsc{NoEvent})$$

$$\frac{\texttt{varsPat } pat = (x_i)^i \qquad Some((v_i)^i) = \rho[x]}{G, \rho \vdash () \xrightarrow[\texttt{let } pat=x?]{step} (), True, \rho[v_i/x_i]^i} \quad (\textsc{Event})$$

4.1.2 Services

A service defines and instantiates a specific feature within the car system. It interacts with the car system through signals and events, which it can import to receive values asynchronously and export to send values asynchronously. The service's definition specifies the behavior of the exported signals and events based on the imported ones using declarative equations. When an imported signal or event receives a value, the service propagates the changes to ensure that its equations remain valid.

Internally, values sent within a signal represent updates to the signal, while values in an event indicate its presence. As a result, a value is meaningful only when paired with its time of occurrence. Additionally, knowing the timing of signals and events enables the definition of timing constraints in equations, using the `timeout` operator for example.

The arrival of signal and event occurrences is managed by a runtime system, as shown in Fig. 4.1. This system receives occurrences, propagates the corresponding equations within the service, and sends updated exports. When multiple values arrive in quick succession, they are stored in a priority queue along with their arrival times, ensuring that older values are propagated before newer ones.

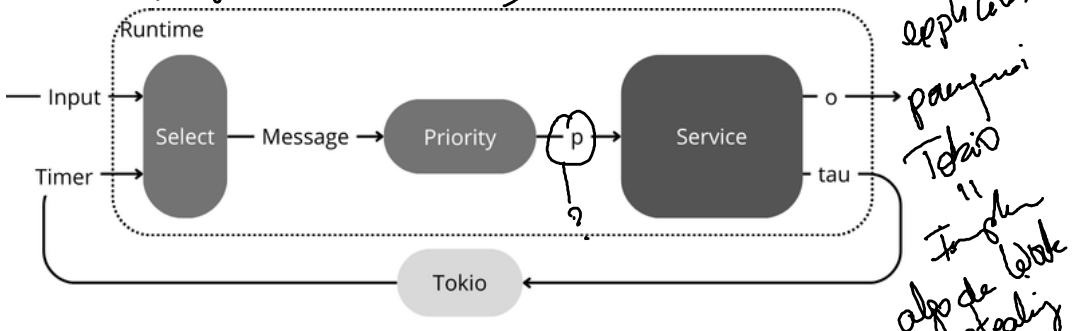


Figure 4.1: The execution of a service is handled by a runtime that processes import occurrences, propagates the corresponding equations within the service, and sends updated exports.

The operational semantics of a service, described bellow, captures the execution of the runtime.

Types Here are the types we consider:

`bvalue` α	absent or present value
`AsyncStream` α	represents async stream
`list` α	list

Notations Here are the notations we use:

`Input` / `Timer` / `Pending`	: `Message`	message can be input from interface, timer or empty
$\perp$ / v	: `bvalue` α	absence / presence of value v
bv	: `bvalue` α	absence or present value
ϵ	: `AsyncStream` α	empty async stream
$v \diamond s$	: `AsyncStream` α	construction of async stream
$[x_i]^i$	: `list` α	list of index $i \in I$ (I is finite and implicit)

Names Here are the names we use:

G : $Ident \to Def$	maps the nodes to their definitions
γ : $Ident \to$ `bvalue` α	maps identifiers to absent or present value

Semantics Here are the properties notations:

$G \vdash (p, s, \tau, o) \xrightarrow[\texttt{service}\ name\ \{(\mathcal{S}_j)^j\}]{\texttt{Message}} (p', s', \tau', o')$	service reacting to a message
$G \vdash (s, \tau, o) \xrightarrow[\texttt{service}\ name\ \{(\mathcal{S}_j)^j\}]{i,t,v} (s', \tau', o')$	service propagating an input
$G \vdash (\gamma, s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o')$	statement propagating an input
$G \vdash (\gamma, s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')$	statement propagating the context
$G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i$	expression returning a value by propagating the contex

Functions Here are helping functions (added to the previous ones):

`insert`	: `AsyncStream` $\alpha \to Ident \times Times \times \alpha$ $\to$ `AsyncStream` α	insert item in priority stream in order
`push`	: `AsyncStream` $\alpha \to Ident \times Times \times \alpha$ $\to$ `AsyncStream` α	push item in output stream
`reset`	: `AsyncStream unit` $\to Ident \times Times$ $\to$ `AsyncStream unit`	reset a timer in timer stream
`intoValue`	: `list` $\alpha \to$ `list bvalue` $\alpha \to$ `list` α	get input values
`intoBotValue`	: `list` $\alpha \to$ `list` $\alpha \to$ `list bvalue` α	get output absent or present values
`getTimers`	: $FlowExpr \to \mathcal{P}(Ident)$	get the timers involved in the expression

Calling `insert` p (i, t, v) adds the item i with value v and timestamp t to the priority stream p such that all items are ordered by increasing time.

`insert` p (i, t, v) : `AsyncStream` α

$$\texttt{insert}\ \epsilon\ (i,t,v) = (i,t,v) \diamond \epsilon$$

$$\texttt{insert}\ ((i',t',v') \diamond p)\ (i,t,v) = \begin{cases} (i',t',v') \diamond (\texttt{insert}\ p\ (i,t,v)) & \text{if } t' < t \\ (i,t,v) \diamond (i',t',v') \diamond p & \text{otherwise} \end{cases}$$

Calling `push` o (x, t, v) appends the item x with value v and timestamp t to the end of the asynchronous stream o.

`push` o (x, t, v) : `AsyncStream` α

$$\texttt{push}\ \epsilon\ (x,t,v) = (x,t,v) \diamond \epsilon$$
$$\texttt{push}\ ((x',t',v') \diamond o)\ (x,t,v) = (x',t',v') \diamond (\texttt{push}\ o\ (x,t,v))$$

Calling `reset` τ (i_T, t) removes the previous occurrence of i_T from the stream τ and inserts a new occurrence at time t, ensuring that the stream τ remains ordered by increasing time.

`reset` τ (i_T, t) : `AsyncStream unit`

$$\texttt{reset}\ \epsilon\ (i_T,t) = (i_T,t,()) \diamond \epsilon$$
$$\texttt{reset}\ ((i_{T'},t',()) \diamond \tau)\ (i_T,t) = \begin{cases} \texttt{reset}\ \tau\ (i_T,t) & \text{if } i_{T'} = i_T \\ (i_{T'},t',()) \diamond (\texttt{reset}\ \tau\ (i_T,t)) & \text{if } t' < t \\ (i_T,t,()) \diamond (i_{T'},t',()) \diamond \tau & \text{otherwise} \end{cases}$$

Calling `intoValue` $(v_i)^{i \in I}$ $(bv_i)^i$ generates a tuple of current values by combining the flow arrivals $(bv_i)^i$ and the last signal values $(v_i)^{i \in I}$, where I is a set of signals. For each i, the i^{th} element is determined as follows. If $bv_i = \bot$: the value is $None$ if $i \notin I$, or v_i if $i \in I$. If $bv_i = v$: the value is $Some(v)$ if $i \notin I$, or v if $i \in I$.

`intoValue` $(v_i)^{i \in I}$ $(bv_i)^i$: `list` α

$$\texttt{intoValue}\ (v_i)^{i \in I}\ (bv_i)^i = (\begin{cases} v & \text{if } i \in I \wedge bv_i = v_\bot \\ v_i & \text{if } i \in I \wedge bv_i = \bot \\ Some(v) & \text{if } i \notin I \wedge bv_i = v_\bot \\ None & \text{if } i \notin I \wedge bv_i = \bot \end{cases})^i$$

Calling `intoBotValue` $(v_j)^{j \in J}$ $(v'_j)^j$ creates a tuple of flow updates by comparing the current values $(v'_j)^j$ with the last signal values $(v_j)^{j \in J}$, where J represents a set of signals. For each j, the j^{th} element is determined as follows. If $j \in J$, the bottom value indicates a signal update, it is $\bot$ if the signal has not changed. If $j \notin J$, the bottom value represents an event occurrence, it is $\bot$ if the value v'_j is $None$.

`intoBotValue` $(v_j)^{j \in J}$ $(v'_j)^j$: `list bvalue` α

$$\texttt{intoBotValue}\ (v_j)^{j \in J}\ (v'_j)^j = (\begin{cases} v_{j\bot} & \text{if } j \in J \wedge v'_j \neq v_j \\ \bot & \text{if } j \in J \wedge v'_j = v_j \\ v_\bot & \text{if } j \notin J \wedge v'_j = Some(v) \\ \bot & \text{if } j \notin J \wedge v'_j = None \end{cases})^j$$

Calling `getTimers` σ returns the set of timer identifiers involved in the flow expression σ.

$\boxed{\texttt{getTimers}\ \sigma\ :\ \mathcal{P}(\mathit{Ident})}$

$$
\begin{aligned}
\texttt{getTimers}\ x &= \emptyset \\
\texttt{getTimers}\ (\sigma_i)^i &= \cup_i\, \texttt{getTimers}\ \sigma_i \\
\texttt{getTimers}\ f(\sigma_i)^i &= \cup_i\, \texttt{getTimers}\ \sigma_i \\
\texttt{getTimers sample}\ \sigma\ T &= \{i_T\}\ \cup\ \texttt{getTimers}\ \sigma \\
\texttt{getTimers scan}\ \sigma\ T &= \{i_T\}\ \cup\ \texttt{getTimers}\ \sigma \\
\texttt{getTimers timeout}\ \sigma\ T &= \{i_T\}\ \cup\ \texttt{getTimers}\ \sigma \\
\texttt{getTimers throttle}\ \sigma\ T &= \texttt{getTimers}\ \sigma \\
\texttt{getTimers onchange}\ \sigma\ T &= \texttt{getTimers}\ \sigma \\
\texttt{getTimers persist}\ \sigma\ T &= \texttt{getTimers}\ \sigma \\
\texttt{getTimers merge}\ \sigma_1\ \sigma_2\ T &= \texttt{getTimers}\ \sigma_1\ \cup\ \texttt{getTimers}\ \sigma_2
\end{aligned}
$$

Receive message by runtime

The property $G \vdash (p, s, \tau, o) \xrightarrow[\texttt{service}\ \mathit{name}\ \{(\mathcal{S}_j)^j\}]{\texttt{Message}} (p', s', \tau', o')$ describes how a service, defined in a program G, reacts to a message `Message`. The runtime managing the service includes the service state s, a priority stream p for buffering inputs, a timer queue τ, and an output queue o.

Message handling is governed by three rules, each corresponding to a specific type of message. Receiving `Pending`, rule (Pending), means there is no more inputs to collect and the service can process items in the priority stream one by one. Handling a message $\texttt{Input}(i, t, v)$ is described by the rule (Input), the input's value and timestamp are inserted in the priority stream p. Rule (Timer) applies when a $\texttt{Timer}(i_T, t)$ message is received, it adds the timer to the priority stream p.

$\boxed{G \vdash (p, s, \tau, o) \xrightarrow[\texttt{service}\ \mathit{name}\ \{(\mathcal{S}_j)^j\}]{\texttt{Message}} (p', s', \tau', o')}$

Prop : P est toujours ordonnée après f° insert

$$\frac{p = (i, t, v) \diamond p' \qquad G \vdash (s, \tau, o) \xrightarrow[\texttt{service}\ \mathit{name}\ \{(\mathcal{S}_j)^j\}]{i,t,v} (s', \tau', o')}{G \vdash (p, s, \tau, o) \xrightarrow[\texttt{service}\ \mathit{name}\ \{(\mathcal{S}_j)^j\}]{\texttt{Pending}} (p', s', \tau', o')} \quad (\textsc{Pending})$$

$$\frac{p' = \texttt{insert}\ p\ (i, t, v)}{G \vdash (p, s, \tau, o) \xrightarrow[\texttt{service}\ \mathit{name}\ \{(\mathcal{S}_j)^j\}]{\texttt{Input}(i,t,v)} (p', s, \tau, o)} \quad (\textsc{Input})$$

$$\frac{p' = \texttt{insert}\ p\ (i_T, t, ())\qquad \texttt{contains}\ \tau\ (i_T, t)}{G \vdash (p, s, \tau, o) \xrightarrow[\texttt{service}\ \mathit{name}\ \{(\mathcal{S}_j)^j\}]{\texttt{Timer}(i_T,t)} (p', s, \tau, o)} \quad (\textsc{Timer})$$

Propagate input in service

The processing of (i,t,v) by a service is denoted as $G \vdash (s,\tau,o) \xrightarrow[\texttt{service}\ name\ (\mathcal{S}_j)^j]{i,t,v} (s',\tau',o')$. This represents the propagation of (i,t,v) through the service's equations $(\mathcal{S}_j)^j$, as defined by rule (Service).

$$\boxed{G \vdash (s,\tau,o) \xrightarrow[\texttt{service}\ name\ \{(\mathcal{S}_j)^j\}]{i,t,v} (s',\tau',o')}$$

$$\frac{G \vdash (\gamma_{empty}, s,\tau,o) \xrightarrow[(\mathcal{S}_j)^j]{i,t,v} (\gamma', s',\tau',o')}{G \vdash (s,\tau,o) \xrightarrow[\texttt{service}\ name\ \{(\mathcal{S}_j)^j\}]{i,t,v} (s',\tau',o')} \quad (\textsc{Service})$$

Propagate input in equations

The propagation of (i,t,v) through the equations $(\mathcal{S}_j)^j$ is written as $G \vdash (\gamma, s,\tau,o) \xrightarrow[(\mathcal{S}_j)^j]{i,t,v} (\gamma', s',\tau',o')$, where γ represents the context of identifiers, which expands as more equations are evaluated.

The first rule, (Skip), represents the termination case of the propagation.

The rule (ImportPresent) applies when the equation to evaluate is an import, and the import corresponds to the propagated input flow i. In this case, the value carried by i is added to the context γ and consumed, with only the updated context being propagated through the remaining equations. The analogous rule (ImportAbsent) applies when the equation to evaluate is an import, but the import does not correspond to the propagated input flow i. In this case, the value carried by i is propagated through the remaining equations without affecting the context.

The propagation through an export equation is defined by two rules, depending on whether the value to export has been computed. In the first case, rule (ExportValue), the value is added to the output stream, and the remaining equations are propagated. In the second case, rule (ExportBottom), only the remaining equations are propagated, as no value is to be exported.

Input signals are handled differently depending on their type. When a signal or event value is imported into the service, it is added to the context and can be used by multiple equations. On the other hand, a timer value is specific to a single flow expression and is only added to the context while evaluating that particular expression, as explained in rule (LetTimer). If the input propagated in the `let` equation is not a timer for the flow expression, rule (LetPropag) applies, and the flow expression is evaluated using the current context.

$$\boxed{G \vdash (\gamma, s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o')}$$

$$\frac{}{G \vdash (\gamma, s, \tau, o) \xrightarrow[\cdot]{i,t,v} (\gamma, s, \tau, o)} \quad (\textsc{Skip})$$

$$\frac{x = i \qquad G \vdash (\gamma + [v_\perp/x], s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')}{G \vdash (\gamma, s, \tau, o) \xrightarrow[\texttt{import } k\ x\ ;\ (\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o')} \quad (\textsc{ImportPresent})$$

$$\frac{x \neq i \qquad G \vdash (\gamma, s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o')}{G \vdash (\gamma, s, \tau, o) \xrightarrow[\texttt{import } k\ x\ ;\ (\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o')} \quad (\textsc{ImportAbsent})$$

$$\frac{\gamma[x] = v_{x\perp} \qquad G \vdash (\gamma, s, \tau, \texttt{push } o\ (x, t, v_x)) \xrightarrow[(\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o')}{G \vdash (\gamma, s, \tau, o) \xrightarrow[\texttt{export } k\ x\ ;\ (\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o')} \quad (\textsc{ExportValue})$$

$$\frac{\gamma[x] = \perp \qquad G \vdash (\gamma, s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o')}{G \vdash (\gamma, s, \tau, o) \xrightarrow[\texttt{export } k\ x\ ;\ (\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o')} \quad (\textsc{ExportBottom})$$

A PRECISER.

$$\frac{\begin{array}{l} i \in \texttt{getTimers } \sigma \qquad G, \gamma + [i/()_\perp] \vdash (s_\sigma, \tau) \xrightarrow[\sigma]{t} (s'_\sigma, \tau_\sigma), [bv_n]^n \\ \texttt{idents } pat = [x_n]^n \qquad \gamma_\sigma = \gamma + [bv_n/x_n]^n \qquad G \vdash (\gamma_\sigma, s, \tau_\sigma, o) \xrightarrow[(\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o') \end{array}}{G \vdash (\gamma, (s_\sigma, s), \tau, o) \xrightarrow[\texttt{let } pat=\sigma\ ;\ (\mathcal{S}_j)^j]{i,t,v} (\gamma', (s'_\sigma, s'), \tau', o')} \quad (\textsc{LetTimer})$$

différent de l'identité ?

$$\frac{\begin{array}{l} i \notin \texttt{getTimers } \sigma \qquad G, \gamma \vdash (s_\sigma, \tau) \xrightarrow[\sigma]{t} (s'_\sigma, \tau_\sigma), [bv_n]^n \\ \texttt{idents } pat = [x_n]^n \qquad \gamma_\sigma = \gamma + [bv_n/x_n]^n \qquad G \vdash (\gamma_\sigma, s, \tau_\sigma, o) \xrightarrow[(\mathcal{S}_j)^j]{i,t,v} (\gamma', s', \tau', o') \end{array}}{G \vdash (\gamma, (s_\sigma, s), \tau, o) \xrightarrow[\texttt{let } pat=\sigma\ ;\ (\mathcal{S}_j)^j]{i,t,v} (\gamma', (s'_\sigma, s'), \tau', o')} \quad (\textsc{LetPropag})$$

Propagate context in equations

When propagating the input (i, t, v) through the equations, the input may be consumed. Afterward, only the updated context, which contains all relevant values, is propagated. For signals, a relevant value is any update to its current value, while for events, the relevant value is the occurrence of the event (with true?)

The propagation of the context is written as $G \vdash (\gamma, s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')$, which is similar to the propagation of the input.

Un exemple serait utile.

The termination rule (Skip) remains unchanged. Only the rule (ImportAbsent), which handles the absence of an import, is retained. The rules for `export` equations, (ExportValue) and (ExportBottom), also remain unchanged. The remaining rule for evaluating flow expressions without a triggered timer is (LetPropag). Détails ?

$$\boxed{G \vdash (\gamma, s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')}$$

$$\frac{}{G \vdash (\gamma, s, \tau, o) \xrightarrow{t} (\gamma, s, \tau, o)} \quad \text{(Skip)}$$

$$\frac{G \vdash (\gamma, s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')}{G \vdash (\gamma, s, \tau, o) \xrightarrow[\texttt{import}\ k\ x\ ;\ (\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')} \quad \text{(ImportAbsent)}$$

$$\frac{\gamma[x] = v_{x\perp} \qquad G \vdash (\gamma, s, \tau, \texttt{push}\ o\ (x, t, v_x)) \xrightarrow[(\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')}{G \vdash (\gamma, s, \tau, o) \xrightarrow[\texttt{export}\ k\ x\ ;\ (\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')} \quad \text{(ExportValue)}$$

$$\frac{\gamma[x] = \perp \qquad G \vdash (\gamma, s, \tau, o) \xrightarrow[(\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')}{G \vdash (\gamma, s, \tau, o) \xrightarrow[\texttt{export}\ k\ x\ ;\ (\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o')} \quad \text{(ExportBottom)}$$

$$\frac{\begin{array}{c} G, \gamma \vdash (s_\sigma, \tau) \xrightarrow[\sigma]{t} (s'_\sigma, \tau_\sigma), [bv_n]^n \\ \texttt{idents}\ pat = [x_n]^n \qquad \gamma_\sigma = \gamma + [bv_n / x_n]^n \qquad G \vdash (\gamma_\sigma, s, \tau_\sigma, o) \xrightarrow[(\mathcal{S}_j)^j]{t} (\gamma', s', \tau', o') \end{array}}{G \vdash (\gamma, (s_\sigma, s), \tau, o) \xrightarrow[\texttt{let}\ pat=\sigma\ ;\ (\mathcal{S}_j)^j]{t} (\gamma', (s'_\sigma, s'), \tau', o')} \quad \text{(LetPropag)}$$

Evaluate flow expression ⟶ GRFRP ? A introduire avant ?

Flow expressions evaluate signals and events using the context γ to retrieve updates, which are represented as bottom values. If the value for a signal stored in γ is $\perp$, it indicates that the signal has not changed. Similarly, if the value for an event stored in γ is $\perp$, it means that the event is absent.

Flow expressions are designed to handle the presence and absence of events, or updates to signals, depending on the expression, to create the appropriate bottom values. Since flow expressions need to track previous values, such as for detecting signal changes, they are stateful expressions. Additionally, some expressions, like `timeout`, create new timers, which must be stored in a timer stream τ.

The evaluation of a flow expression σ in the context γ is written as $G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i$. In this notation, s and s' represent the current and next states, τ and τ' are the current and updated timer streams, and $[bv_i]^i$ are the evaluated bottom values.

As described in rule (Ident), the identifier expression (x) returns the value stored in the context, independently of the flow kind (signal or event).

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *flow identifier*

$$\frac{\gamma(x) = bv}{G, \gamma \vdash ((), \tau) \xrightarrow[x]{t} ((), \tau), [bv]} \quad (\textsc{Ident})$$

The expression `sample` $\sigma\ T$ creates an event that periodically returns the most recent value of the event expression σ, provided that σ has occurred. This allows control over when the event occurs, as it determines the timing of the event. To achieve this, the expression needs to store the previous value of σ in memory, represented as a bottom value bv. If σ has not occurred since the last period, bv is set to $\bot$.

When σ has a value, the rule (SampleValue) is applied to update the memory bv with the new value.

If σ returns $\bot$, indicating that the event is absent, the rule (SamplePropag) propagates the new state s' of σ.

If the propagation is triggered by a timer i_T associated with the `sample` expression, the rule (SampleTimer) states that the expression evaluates to the stored value bv, the memory is then reset to $\bot$, and the corresponding timer in the timer stream τ is also reset.

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *event sampling*

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [v'_\bot]}{G, \gamma \vdash ((bv, s), \tau) \xrightarrow[\texttt{sample}\ \sigma\ T]{t} ((v'_\bot, s'), \tau'), [\bot]} \quad (\textsc{SampleValue})$$

$$\frac{\gamma[i_T] = ()_\bot}{G, \gamma \vdash ((bv, s), \tau) \xrightarrow[\texttt{sample}\ \sigma\ T]{t} ((\bot, s), \texttt{reset}\ \tau\ (i_T, t + T)), [bv]} \quad (\textsc{SampleTimer})$$

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [\bot]}{G, \gamma \vdash ((bv, s), \tau) \xrightarrow[\texttt{sample}\ \sigma\ T]{t} ((bv, s'), \tau'), [\bot]} \quad (\textsc{SamplePropag})$$

The flow expression `scan` $\sigma\ T$ generates a signal that only includes the value updates from σ at each period T. It stores the most recent value update of σ named bv, as well as the last value `scan` $\sigma\ T$ has been evaluated to named v_l.

When σ has a value different from v_l, the rule (ScanValue) is applied to update the memory bv with the new value. If, however, the value of σ is found to be equal to v_l, the rule (ScanDrop) resets bv to $\bot$, indicating that no update has been made to the signal `scan` $\sigma\ T$.

If σ returns $\bot$, indicating that the signal is unchanged, the rule (ScanPropag) propagates the new state s' of σ.

If the propagation is triggered by a timer i_T associated with the `scan` expression and the stored value is $v_\bot$, the rule (ScanTimer) specifies that the expression evaluates to $v_\bot$. In this case, the memory is reset to $\bot$, the last value is updated to v, and the corresponding timer in the timer stream τ is also reset. If, instead, the stored value is $\bot$, the rule (ScanBot) indicates that the expression evaluates to $\bot$. Here, the memory is reset to $\bot$, the last value remains unchanged, and the corresponding timer in the timer stream τ is reset.

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *signal scan*

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [v'_\perp] \qquad v' \neq v_l}{G, \gamma \vdash ((bv, v_l, s), \tau) \xrightarrow[\texttt{scan}\ \sigma\ T]{t} ((v'_\perp, v_l, s'), \tau'), [\perp]} \quad (\textsc{ScanValue})$$

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [v'_\perp] \qquad v' = v_l}{G, \gamma \vdash ((bv, v_l, s), \tau) \xrightarrow[\texttt{scan}\ \sigma\ T]{t} ((\perp, v_l, s'), \tau'), [\perp]} \quad (\textsc{ScanDrop})$$

$$\frac{\gamma[i_T] = ()_\perp}{G, \gamma \vdash ((\perp, v_l, s), \tau) \xrightarrow[\texttt{scan}\ \sigma\ T]{t} ((\perp, v_l, s), \texttt{reset}\ \tau\ (i_T, t + T)), [\perp]} \quad (\textsc{ScanBot})$$

$$\frac{\gamma[i_T] = ()_\perp}{G, \gamma \vdash ((v_\perp, v_l, s), \tau) \xrightarrow[\texttt{scan}\ \sigma\ T]{t} ((\perp, v, s), \texttt{reset}\ \tau\ (i_T, t + T)), [v_\perp]} \quad (\textsc{ScanTimer})$$

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [\perp]}{G, \gamma \vdash ((bv, v_l, s), \tau) \xrightarrow[\texttt{scan}\ \sigma\ T]{t} ((bv, v_l, s'), \tau'), [\perp]} \quad (\textsc{ScanPropag})$$

The flow expression `timeout` $\sigma\ T$ creates an event with unit values, triggered when σ has not returned a value for more than T seconds.

When σ has a value, the rule (TimeoutReset) is applied to reset the corresponding timer in the timer stream τ.

If σ returns $\perp$, indicating the event is absent, the rule (TimeoutPropag) propagates the new state s' of σ.

If the propagation is triggered by a timer i_T associated with the `timeout` expression, the rule (TimeoutTimer) specifies that the expression evaluates to $()_\perp$, and the corresponding timer in the timer stream τ is reset.

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *event timeout*

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [v'_\perp]}{G, \gamma \vdash (s, \tau) \xrightarrow[\texttt{timeout}\ \sigma\ T]{t} (s', \texttt{reset}\ \tau'\ (i_T, t + T)), [\perp]} \quad (\textsc{TimeoutReset})$$

$$\frac{\gamma[i_T] = ()_\perp}{G, \gamma \vdash (s, \tau) \xrightarrow[\texttt{timeout}\ \sigma\ T]{t} (s, \texttt{reset}\ \tau\ (i_T, t + T)), [()_\perp]} \quad (\textsc{TimeoutTimer})$$

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [\perp]}{G, \gamma \vdash (s, \tau) \xrightarrow[\texttt{timeout}\ \sigma\ T]{t} (s', \tau'), [\perp]} \quad (\textsc{TimeoutPropag})$$

The flow expression `throttle` σ Δ creates a signal that retains only the values from σ whose difference from the previous value is greater than Δ. To evaluate this expression, the last value, denoted v_l, must be stored.

When σ has a value such that the difference from v_l is greater than Δ, the rule (ThrottleValue) is applied, updating the expression to this new value and storing it as the new v_l. If the difference from v_l is smaller than Δ, the rule (ThrottleDrop) is applied to discard the new value, setting the expression to $\bot$.

If σ returns $\bot$, indicating that the signal remains unchanged, the rule (ThrottlePropag) propagates the new state s' of σ.

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *signal throttle*

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [v'_\bot] \qquad |v' - v_l| \geq \Delta}{G, \gamma \vdash ((v_l, s), \tau) \xrightarrow[\texttt{throttle}\ \sigma\ \Delta]{t} ((v', s'), \tau'), [v'_\bot]} \quad \text{(ThrottleValue)}$$

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [v'_\bot] \qquad |v' - v_l| < \Delta}{G, \gamma \vdash ((v_l, s), \tau) \xrightarrow[\texttt{throttle}\ \sigma\ \Delta]{t} ((v_l, s'), \tau'), [\bot]} \quad \text{(ThrottleDrop)}$$

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [\bot]}{G, \gamma \vdash ((v_l, s), \tau) \xrightarrow[\texttt{throttle}\ \sigma\ \Delta]{t} ((v_l, s'), \tau'), [\bot]} \quad \text{(ThrottlePropag)}$$

The flow expression `onchange` σ creates an event derived from a signal. The actual values of the flow remain unchanged, as indicated by the rules (OnchangeValue) and (OnchangePropag), but their interpretation and behavior when used in other flow expressions are modified.

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *signal changes*

ajouter 1 exp à chaque fois

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [v'_\bot]}{G, \gamma \vdash (s, \tau) \xrightarrow[\texttt{onchange}\ \sigma]{t} (s', \tau'), [v'_\bot]} \quad \text{(OnchangeValue)}$$

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [\bot]}{G, \gamma \vdash (s, \tau) \xrightarrow[\texttt{onchange}\ \sigma]{t} (s', \tau'), [\bot]} \quad \text{(OnchangePropag)}$$

The flow expression `persist` σ derives a signal from an event. Since the values of a signal represent its updates, converting an event into a signal requires filtering out redundant event values. This behavior is captured by the rule (PersistDrop). In other cases, the values of the flow remain unchanged, as demonstrated by the rules (PersistValue) and (PersistPropag). However, their interpretation and behavior when used in other flow expressions are modified, mirroring the changes made by the `onchange` expression in the opposite direction.

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *event persistance*

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [v'_\perp] \qquad v' \neq v_l}{G, \gamma \vdash ((v_l, s), \tau) \xrightarrow[\texttt{persist}\ \sigma]{t} ((v', s'), \tau'), [v'_\perp]} \quad (\textsc{PersistValue})$$

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [v'_\perp] \qquad v' = v_l}{G, \gamma \vdash ((v_l, s), \tau) \xrightarrow[\texttt{persist}\ \sigma]{t} ((v_l, s'), \tau'), [\perp]} \quad (\textsc{PersistDrop})$$

$$\frac{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [\perp]}{G, \gamma \vdash ((v_l, s), \tau) \xrightarrow[\texttt{persist}\ \sigma]{t} ((v_l, s'), \tau'), [\perp]} \quad (\textsc{PersistPropag})$$

The flow expression `merge` σ_1 σ_2 combines the values from the events σ_1 and σ_2 into a new event. In cases where both events occur simultaneously, as handled by rule (MergeValue12), the resulting event prioritizes the values from σ_1. If only one event occurs, rules (MergeValue1) or (MergeValue2) are applied, depending on which event occurred, and the value of this event is returned. When both events return $\perp$, indicating their absence, rule (MergePropag) propagates the updated states s'_1 and s'_2 of σ_1 and σ_2, respectively.

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *events merge*

$$\frac{G, \gamma \vdash (s_1, \tau) \xrightarrow[\sigma_1]{t} (s'_1, \tau'), [v_{1\perp}] \qquad G, \gamma \vdash (s_2, \tau) \xrightarrow[\sigma_2]{t} (s'_2, \tau'), [v_{2\perp}]}{G, \gamma \vdash ((s_1, s_2), \tau) \xrightarrow[\texttt{merge}\ \sigma_1\ \sigma_2]{t} ((s'_1, s'_2), \tau'), [v_{1\perp}]} \quad (\textsc{MergeValue12})$$

$$\frac{G, \gamma \vdash (s_1, \tau) \xrightarrow[\sigma_1]{t} (s'_1, \tau'), [v_{1\perp}] \qquad G, \gamma \vdash (s_2, \tau) \xrightarrow[\sigma_2]{t} (s'_2, \tau'), [\perp]}{G, \gamma \vdash ((s_1, s_2), \tau) \xrightarrow[\texttt{merge}\ \sigma_1\ \sigma_2]{t} ((s'_1, s'_2), \tau'), [v_{1\perp}]} \quad (\textsc{MergeValue1})$$

$$\frac{G, \gamma \vdash (s_1, \tau) \xrightarrow[\sigma_1]{t} (s'_1, \tau'), [\perp] \qquad G, \gamma \vdash (s_2, \tau) \xrightarrow[\sigma_2]{t} (s'_2, \tau'), [v_{2\perp}]}{G, \gamma \vdash ((s_1, s_2), \tau) \xrightarrow[\texttt{merge}\ \sigma_1\ \sigma_2]{t} ((s'_1, s'_2), \tau'), [v_{2\perp}]} \quad (\textsc{MergeValue2})$$

$$\frac{G, \gamma \vdash (s_1, \tau) \xrightarrow[\sigma_1]{t} (s'_1, \tau'), [\perp] \qquad G, \gamma \vdash (s_2, \tau) \xrightarrow[\sigma_2]{t} (s'_2, \tau'), [\perp]}{G, \gamma \vdash ((s_1, s_2), \tau) \xrightarrow[\texttt{merge}\ \sigma_1\ \sigma_2]{t} ((s'_1, s'_2), \tau'), [\perp]} \quad (\textsc{MergePropag})$$

The flow expression `time` represents time as a signal. During its execution, it returns only the timestamp of the current propagation, as specified by the rule (Time).

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *time signal*

$$\frac{}{G, \gamma \vdash ((), \tau) \xrightarrow[\texttt{time}]{t} ((), \tau), [t_\perp]} \quad (\textsc{Time})$$

The node application $f(\sigma_i)^i$ creates a tuple of signals and events, determined by the specification of the node f in G.

The state machine associated with f performs a step only when one of its input signals has changed or one of its input events has occurred, as depicted in rule (NodeValue). The service must deliver the current values of all inputs, denoted $(iv'_i)^i$, to the node state machine. These inputs include optional values for events and the most recent values for signals. If a signal has not changed, the service uses the previously stored value from the tuple $(iv_i)^{i \in I}$, where I represents the set of input signals. The node state machine produces output values $(ov'_j)^j$, which the service filters to retain only signal changes and event occurrences, represented as $(obv_j)^j$. To distinguish signal changes, the service stores the previous values of output signals in the tuple $(ov_j)^{j \in J}$, where J represents the set of output signals.

When all events and signals return $\bot$, indicating that they are absent or unchanged, the rule (NodePropag) is applied to propagate the updated states.

$\boxed{G, \gamma \vdash (s, \tau) \xrightarrow[\sigma]{t} (s', \tau'), [bv_i]^i}$ *node application*

$$\frac{\begin{array}{c}\forall i \quad G, \gamma \vdash (s_i, \tau_i) \xrightarrow[\sigma_i]{t} (s'_i, \tau_{i+1}), [ibv_i] \qquad \exists i \quad ibv_i \neq \bot \\ G \vdash s_f, (iv'_i)^i \xrightarrow[f]{step} s'_f, (ov'_j)^j \\ (iv'_i)^i = \texttt{intoValue}\ (iv_i)^{i \in I}\ (ibv_i)^i \qquad (obv_j)^j = \texttt{intoBotValue}\ (ov_j)^{j \in J}\ (ov'_j)^j\end{array}}{G, \gamma \vdash (((s_f, (iv_i)^{i \in I}, (ov_j)^{j \in J}), (s_i)^i), \tau) \xrightarrow[f(\sigma_i)^i]{t} (((s'_f, (iv'_i)^{i \in I}, (ov'_j)^{j \in J}), (s'_i)^i), \tau'), [obv_j]^j} \quad \text{(NodeValue)}$$

$$\frac{\forall i \quad G, \gamma \vdash (s_i, \tau_i) \xrightarrow[\sigma_i]{t} (s'_i, \tau_{i+1}), [\bot]}{G, \gamma \vdash (((s_f, (iv_i)^{i \in I}, (ov_j)^{j \in J}), (s_i)^i), \tau) \xrightarrow[f(\sigma_i)^i]{t} (((s_f, (iv_i)^{i \in I}, (ov_j)^{j \in J}), (s'_i)^i), \tau'), [\bot]^j} \quad \text{(NodePropag)}$$

4.2 Compilation Scheme

4.3 Implementation

4.4 Denotational Semantics

We will first present the denotational semantics of components and then the semantics of services.

4.4.1 Components

kahn semantics inspired from [1]

4.4.2 Services

4.5 Equivalence Theorem

4.6 Safety properties

4.6.1 Determinism

4.6.2 Productivity

4.7 TODO ou Mini plan

- operational semantics (two execution modes)
- compilation scheme
- implementation choices (tokio, creusot)
- denotational semantics
- equivalence property
- safety properties (determinism, productivity)

Chapter 5

Parallelisation

- syntax
- related works
- rust and memory separation with lifetimes
- parallelisation algorithm
- benchmarks (overhead vs optimization)

Chapter 6

Conclusion

Bibliography

[1] Timothy Bourke, Basile Pesin, and Marc Pouzet. Verified compilation of synchronous dataflow with state machines. *ACM Trans. Embed. Comput. Syst.*, 22(5s):137:1–137:26, 2023.

[2] Timothy Bourke and Marc Pouzet. Zélus: a synchronous language with odes. In Calin Belta and Franjo Ivancic, editors, *Proceedings of the 16th international conference on Hybrid systems: computation and control, HSCC 2013, April 8-11, 2013, Philadelphia, PA, USA*, pages 113–118. ACM, 2013.

[3] Albert Cohen, Léonard Gérard, and Marc Pouzet. Programming parallelism with futures in lustre. In Ahmed Jerraya, Luca P. Carloni, Florence Maraninchi, and John Regehr, editors, *Proceedings of the 12th International Conference on Embedded Software, EMSOFT 2012, part of the Eighth Embedded Systems Week, ESWeek 2012, Tampere, Finland, October 7-12, 2012*, pages 197–206. ACM, 2012.

[4] Jean-Louis Colaço, Michael Mendler, Baptiste Pauget, and Marc Pouzet. A constructive state-based semantics and interpreter for a synchronous data-flow language with state machines. *ACM Trans. Embed. Comput. Syst.*, 22(5s):152:1–152:26, 2023.

[5] Jean-Louis Colaço, Bruno Pagano, and Marc Pouzet. SCADE 6: A formal language for embedded critical software development (invited paper). In Frédéric Mallet, Min Zhang, and Eric Madelaine, editors, *11th International Symposium on Theoretical Aspects of Software Engineering, TASE 2017, Sophia Antipolis, France, September 13-15, 2017*, pages 1–11. IEEE Computer Society, 2017.

[6] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. Creusot: A foundry for the deductive verification of rust programs. In Adrián Riesco and Min Zhang, editors, *Formal Methods and Software Engineering - 23rd International Conference on Formal Engineering Methods, ICFEM 2022, Madrid, Spain, October 24-27, 2022, Proceedings*, volume 13478 of *Lecture Notes in Computer Science*, pages 90–105. Springer, 2022.

[7] Conal M. Elliott. Push-pull functional reactive programming. In Stephanie Weirich, editor, *Proceedings of the 2nd ACM SIGPLAN Symposium on Haskell, Haskell 2009, Edinburgh, Scotland, UK, 3 September 2009*, pages 25–36. ACM, 2009.

[8] Nicolas Halbwachs, Paul Caspi, Pascal Raymond, and Daniel Pilaud. The synchronous data flow programming language LUSTRE. *Proc. IEEE*, 79(9):1305–1320, 1991.

Appendix A

Appendix Title